

Moderador Automático de Contenido en Videos

Transcripción, Etiquetado y Clasificación de Contenido Inapropiado

Grupo 4:

Koc Góngora, Luis Enrique
Mancilla Antay, Alex Felipe
Melendez Garcia, Herbert Antonio
Paitan Cano, Dennis Jack

Maestría en Inteligencia Artificial
Universidad Nacional de Ingeniería (UNI)
Procesamiento de Lenguaje Natural

Semestre 2026-1

Desafío Global

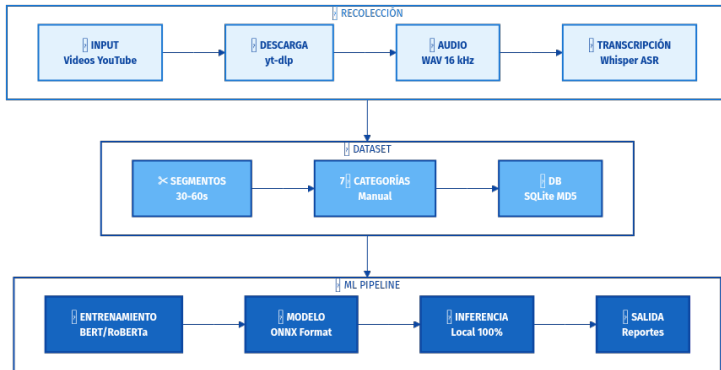
- Plataformas como YouTube, TikTok generan **500+ horas de video por minuto**
- Contenido en idioma español crece exponencialmente en creadores peruanos
- Moderación manual es **costosa, lenta e inconsistente**

Brecha de Investigación

1. No existen soluciones locales para moderación de contenido en español (Perú)
2. Modelos LLM externos = privacidad limitada y costos operacionales altos
3. **Propuesta:** Sistema end-to-end local para transcripción y clasificación automática

Objetivo

Desarrollar un moderador automático que funcione **100 % localmente** sin depender de APIs externas



Pipeline

1. Descarga de video (yt-dlp)
2. Procesamiento audio
3. Transcripción (Whisper)
4. Segmentación
5. Etiquetado manual
6. Limpieza datos
7. Entrenamiento
8. Exportación (ONNX)
9. Inferencia local
10. Reportes

Especificaciones Técnicas

- **Duración:** 50–100 horas de audio ($\approx$ 1000 videos 5 min)
- **Framerate:** 16 kHz (estándar ASR)
- **Canales:** Política, Farándula, Comida, Viajes peruanos
- **Formato output:** WAV mono a 16kHz

Arquitectura Python

- **Herramienta:** yt-dlp (descendiente youtube-dl)
- **Flujo:**
 1. Scraping URLs de canales peruanos
 2. Descarga mejor calidad audio
 3. Conversión a WAV 16kHz mono
 4. Almacenamiento en carpetas categorizadas
- **Manejo errores:** Reintentos con backoff
- **Storage:** Carpetas por categoría/fecha

Pseudocódigo

```
import yt_dlp

ydl = yt_dlp.YoutubeDL({
    'format': 'bestaudio',
    'audio-format': 'wav',
    'audio-quality': '192',
    'quiet': False
})

for url in url_list:
    ydl.download([url])
```

Modelos Recomendados (Local) [Radford et al., 2022]

- **Whisper OpenAI (base/small):** ~81M params, ~1.5GB RAM, excelente en español
- **Wav2Vec2 Facebook:** ~317M params, entrenado multilingüe
- **xlsr-wav2vec2-spanish:** Optimizado para español peruano

Justificación

- **Whisper:** Robusto a ruido y acentos, mejor precisión en español
- **Wav2Vec2:** Menor consumo de memoria, inferencia rápida
- Todos ejecutables localmente en GPU modesta (RTX 3060+)

Métricas esperadas

WER (Word Error Rate) en español: 8–15 % para Whisper base

Categorías de Moderación (7 etiquetas)

1. **Lenguaje Inapropiado** (insultos, groserías)
2. **Racismo/Discriminación** (xenofobia, prejuicio étnico)
3. **Sexismo** (misoginia, discriminación de género)
4. **Contenido Sexual Explícito** (referencias sexuales, acoso)
5. **Violencia** (amenazas, incitación a violencia)
6. **Desinformación/Odio** (fake news, incitación a odio)
7. **Limpio** (sin violación de moderación)

Interfaz Propuesta

HTML5 + JavaScript local: cargar chunks, selector de categoría (botones), persistencia en LocalStorage JSON

Estructura Incremental

- Base datos: **SQLite** (local, sin servidor)
- Schema: (chunk_id, texto, etiqueta, timestamp, hash_MD5)
- Hash MD5 del texto para detectar duplicados

Verificación de Consistencia

1. Antes de insertar: verificar si chunk existe (por hash)
2. Si duplicado: comparar etiqueta, reportar inconsistencias
3. Versioning: mantener histórico de cambios
4. Output limpio: JSON con balanceo de clases

Resultado Final

Dataset de $\approx$ 5000–10000 chunks etiquetados, balanceado, sin duplicados, listo para entrenamiento

Propuesta: Ensemble de Modelos [Devlin et al., 2018]

1. **BERT (Transformer):** Transfer learning, SOTA para clasificación
 - Fine-tuning en dataset etiquetado
 - Precisión esperada: 88–92 %
2. **SVM (Clásico):** Baseline robusto, menos datos necesarios
 - Vectorización: TF-IDF + Word2Vec
 - Precisión esperada: 75–82 %
3. **RoBERTa:** Mejora sobre BERT, mejor en toxicity detection [Liu et al., 2019]

Herramienta Recomendada

Hugging Face Transformers + PyTorch para Transformers; **scikit-learn** para SVM

Métricas Propuestas

- **Accuracy:** Métrica global
- **Precision/Recall:** Por categoría (especialmente clases raras)
- **F1-Score:** Balance entre precisión y recall
- **ROC-AUC:** Curva característica para categorías críticas

Validación

1. Train: 70 % del dataset
2. Validation: 15 %
3. Test: 15 % (hold-out para evaluación final)
4. Cross-validation: 5-fold para robustez

Justificación [Bishop, 2006]

Split 70–15–15 es estándar en NLP y ML

Arquitectura

- **Backend:** Flask/FastAPI (Python)
- **Modelos:** Cargados en memoria (ONNX para optimización)
- **Frontend:** Streamlit o Dashboard React simple
- **Base de datos:** SQLite (local)

Pipeline de Inferencia

1. Input: URL de video o archivo de audio
2. Descargar/procesar audio
3. Transcripción con Whisper
4. Segmentación en chunks (30–60 segundos)
5. Predicción: pasar cada chunk por modelo BERT fine-tuned
6. Output: JSON con predicciones y confianza

Ventajas

100 % local, sin dependencias externas, privacidad garantizada

Plataformas de Contenido

- Creadores de YouTube/TikTok peruanos
- Moderación automática pre-publicación
- Análisis retrospectivo de contenido

Instituciones

- Ministerios (monitoreo de desinformación)
- Universidades (investigación en NLP español)
- ONGs (detección de acoso/ciberabuso)

Impacto

Solución local, privacidad garantizada, adaptable a contexto peruano

Limitaciones Actuales

- Dataset limitado ($\approx$ 5000–10000 chunks)
- Requiere GPU para inferencia rápida
- Dependencia de calidad de transcripción Whisper
- Sesgo en categorías minoritarias

Mejoras Futuras

1. Aumentar dataset a 50K+ chunks
2. Fine-tuning de Whisper específico para español peruano
3. Modelos multimodales (audio + video)
4. Explainability (LIME/SHAP) para decisiones
5. API REST completamente funcional

Fase	Duración	Mes
A.1 Descarga de datos	1.5 semanas	Junio
A.2 Transcripción ASR	1.5 semanas	Junio
A.3 Etiquetado manual	3 semanas	Junio–Julio
A.4 Dataset limpio	1 semana	Julio
B Entrenamiento	2 semanas	Julio
C Framework producción	1.5 semanas	Julio
Documentación + Demo	1 semana	Julio

Total: ~ 12 semanas de trabajo integrado



Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022).
Robust Speech Recognition via Large-Scale Weak Supervision.
arXiv preprint arXiv:2212.04356.



Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018).
BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
arXiv preprint arXiv:1810.04805.



Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., &
Stoyanov, V. (2019).
RoBERTa: A Robustly Optimized BERT Pretraining Approach.
arXiv preprint arXiv:1907.11692.



Bishop, C. M. (2006).
Pattern Recognition and Machine Learning.
Springer Science+Business Media.



Pitsilis, V., Ramampiaro, H., & Langseth, H. (2018).
Detecting Offensive Language in Tweets Using Deep Learning.
arXiv preprint arXiv:1801.04354.



Ribeiro, M. T., Wu, T., Guestrin, C., & Singh, A. (2020).
Beyond Accuracy: Behavioral Testing of NLP Models with CheckList.
In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics,
4902–4912.